molchemist

Molecule rendering from Molfile, SDF, and SMILES

v0.1.2 2026-06-19 MIT

Render chemical structures from Molfile, SDF, and SMILES data.

Eito Yoneyama

`molchemist` renders chemical structures from Molfile / SDF data. It can also parse SMILES strings by generating a 2D layout first. The package turns molecular graphs into an `alchemist` drawing program and renders the final figure with CeTZ.

The package is aimed at Typst documents that need compact molecule figures, publication-oriented skeletal formulae, and light annotation without leaving Typst.

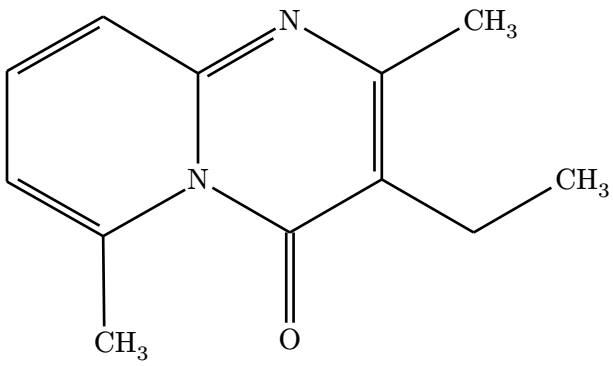
Table of Contents

Part I

Getting Started

Import `molchemist` and choose the renderer that matches your input: `#render-mol` for Molfile or SDF text, and `#render-smiles` for inline SMILES text.

```
#import "@preview/molchemist:0.1.2": *

#let mol-data = read("Structure2D_COMPOUND_CID_93406.sdf")
#render-mol(mol-data, abbreviate: true)
```

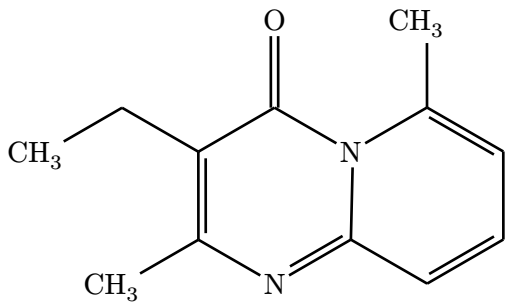


The examples below assume this import unless they need an additional package such as CeTZ.

On Typst 0.15.0 and later, `#render-mol` can also receive `path("molecule.sdf")` directly. This manual keeps using `read(...)` in examples for compatibility with older Typst versions.

SMILES is useful for compact inline examples or generated documents. Because SMILES stores connectivity rather than drawing coordinates, `molchemist` first computes a 2D layout and then renders the structure.

```
#render-smiles(
  "CCC1=C(N=C2C=CC=C(N2C1=O)C)C",
  abbreviate: true,
)
```



I.1 Choosing an Input Format

- `Molfile / SDF`: coordinate-bearing structure text. Use it for database exports or drawing-tool output when preserving the supplied layout matters.
- `SMILES`: compact inline source text. Use it for examples, generated documents, and quick sketches. `molchemist` computes 2D coordinates before rendering.

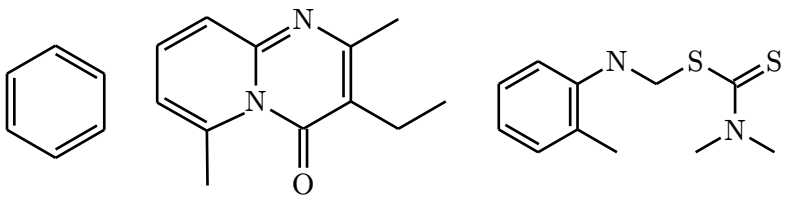
Part II

Example Data

The SDF examples in this manual use real PubChem records included in the repository test data.

- PubChem CID 241[1]: benzene.
- PubChem CID 93406[2]: `3-ethyl-2,6-dimethylpyrido[1,2-a]pyrimidin-4-one`.
- PubChem SID 93298[3]: a deposited DTP/NCI substance record associated with CID 235403.

```
#let benzene = read("Structure2D_COMPOUND_CID_241.sdf")
#let fused = read("Structure2D_COMPOUND_CID_93406.sdf")
#let deposited = read("DepositedStructure_SUBSTANCE_SID_93298_Version_3.sdf")

#grid(
  columns: 3,
  gutter: 7mm,
  align: horizon + center,
  render-mol(benzene, skeletal: true, config: (atom-sep: 1.55em)),
  render-mol(fused, skeletal: true, config: (atom-sep: 1.55em)),
  render-mol(deposited, skeletal: true, config: (atom-sep: 1.55em)),
)
```



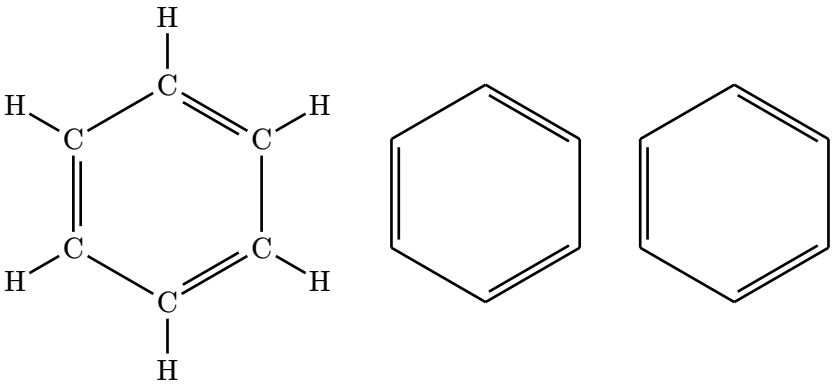
[1] https://pubchem.ncbi.nlm.nih.gov/compound/241
[2] https://pubchem.ncbi.nlm.nih.gov/compound/93406
[3] https://pubchem.ncbi.nlm.nih.gov/substance/93298

Part III

Rendering Modes

Both renderers support the same three modes. Full mode is useful for small molecules and debugging. Abbreviated mode folds common hydrogens and terminal carbons into labels. Skeletal mode is usually the best default for publication figures.

```
#let mol-data = read("Structure2D_COMPOUND_CID_241.sdf")

#grid(
  columns: 3,
  gutter: 8mm,
  align: horizon + center,
  render-mol(mol-data),
  render-mol(mol-data, abbreviate: true),
  render-mol(mol-data, skeletal: true),
)
```

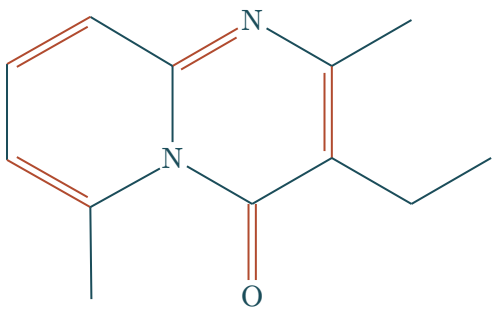


Part IV

Styling

Pass `⟨config⟩` for visual adjustments that should reach `alchemist`, such as atom spacing, fragment color, and bond styles.

```
#let mol-data = read("Structure2D_COMPOUND_CID_93406.sdf")

#render-mol(
  mol-data,
  skeletal: true,
  config: (
    atom-sep: 2.6em,
    fragment-color: rgb("#1b4d5a"),
    single: (stroke: 0.75pt + rgb("#1b4d5a")),
    double: (stroke: 0.75pt + rgb("#b14b2f")),
  ),
)
```



> Molfile and SDF input already contains coordinates. Routing-oriented `alchemist` settings do not reshape that graph; prefer `atom-sep`, font, and stroke settings for visual tuning.

Part V

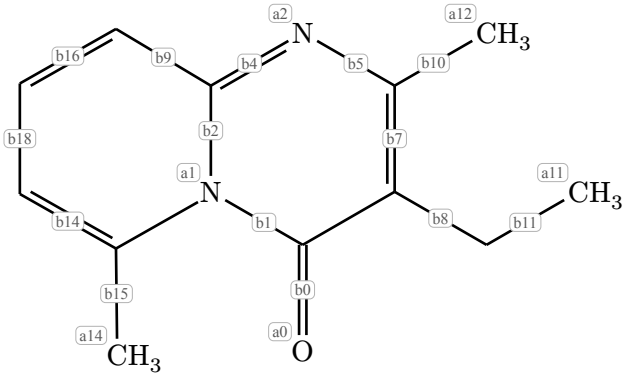
Annotations

Annotations are drawn after the molecule. They are meant for sparse publication callouts, not for replacing a full figure editor.

V.1 Finding Atom and Bond Indices

Enable `⟨show-indices⟩` while authoring. The visible labels are the indices used by `#atom-anchor` and `#bond-anchor`.

```
#let mol-data = read("Structure2D_COMPOUND_CID_93406.sdf")
#render-mol(mol-data, abbreviate: true, show-indices: true)
```

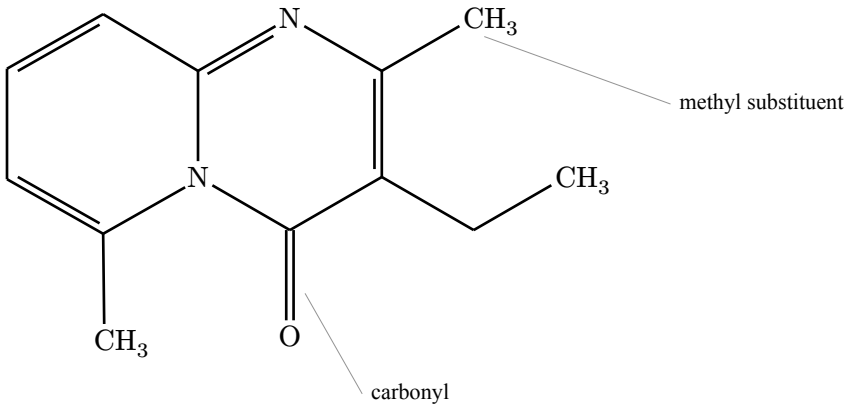


V.2 Callouts

Use `#callout-annotation` for external labels. Defaults are intentionally quiet: no arrowhead, a thin leader line, and an unboxed label.

```
#let mol-data = read("Structure2D_COMPOUND_CID_93406.sdf")

#render-mol(
  mol-data,
  abbreviate: true,
  annotations: (
    callout-annotation(
      bond-anchor(0),
      [carbonyl],
      label-at: (to: molecule-anchor(anchor: "south"), rel: (0.82, -0.48)),
      leader-start: (to: molecule-anchor(anchor: "south"), rel: (0.7, -0.48)),
      leader-end: (to: bond-anchor(0), rel: (0.2, -0.24)),
      leader: "curve",
      stroke: luma(58%) + 0.28pt,
```

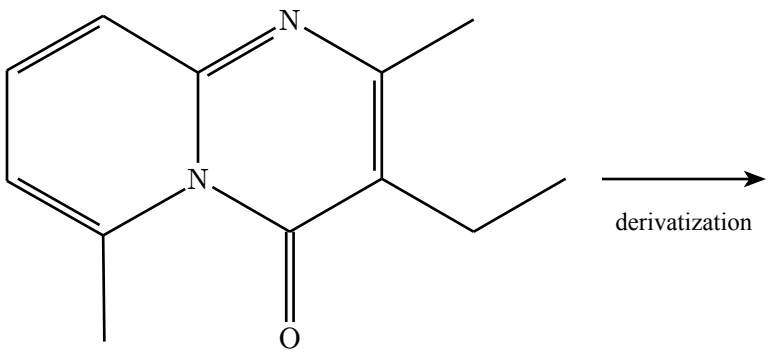
```
      label-size: 0.76em,
      label-anchor: "west",
    ),
    callout-annotation(
      atom-anchor(12),
      [methyl substituent],
      label-at: (to: molecule-anchor(anchor: "east"), rel: (0.9, 1.04)),
      leader-start: (to: molecule-anchor(anchor: "east"), rel: (0.78, 1.04)),
      leader-end: (to: atom-anchor(12), rel: (0.28, -0.38)),
      leader: "curve",
      stroke: luma(58%) + 0.28pt,
      label-size: 0.76em,
      label-anchor: "west",
    ),
  ),
)
```



V.3 Arrows

Use #arrow-annotation for molecule-level process arrows or simple directional marks.

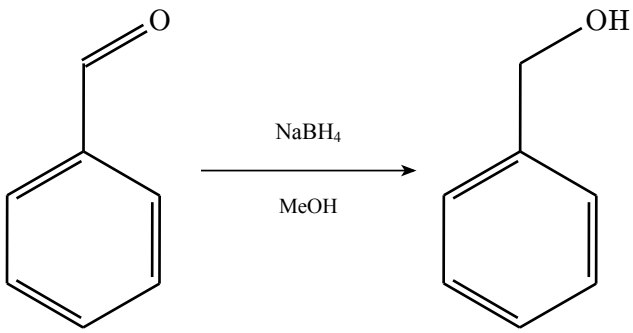
```
#let mol-data = read("Structure2D_COMPOUND_CID_93406.sdf")

#render-mol(
  mol-data,
  skeletal: true,
  annotations: arrow-annotation(
    (to: molecule-anchor(anchor: "east"), rel: (0.48, 0)),
    (to: molecule-anchor(anchor: "east"), rel: (2.65, 0)),
    label: [derivatization],
    label-offset: (0, -0.46),
    label-anchor: "north",
  ),
)
```



V.4 Reaction Schemes

For reactions, compose separate molecule renderings with normal Typst/CeTZ layout. This keeps reaction arrows independent from molecule annotations.

```
#let reaction-arrow(above, below: none) = cetz.canvas({
  import cetz.draw: *
  line(
    (0.1, 0),
    (2.95, 0),
    stroke: 0.65pt + black,
    mark: (end: ">>", scale: 0.72, fill: black),
  )
  content((1.52, 0.42), text(size: 0.78em)[#above], anchor: "south")
  if below != none {
    content((1.52, -0.38), text(size: 0.72em)[#below], anchor: "north")
  }
})

#grid(
  columns: (auto, 28mm, auto),
  column-gutter: 4mm,
  align: horizon + center,
  render-smiles("O=CC1=CC=CC=C1", abbreviate: true),
  reaction-arrow([NaBH#sub[4]], below: [MeOH]),
  render-smiles("OCC1=CC=CC=C1", abbreviate: true),
)
```



V.5 Mechanism Example: von Richter Reaction

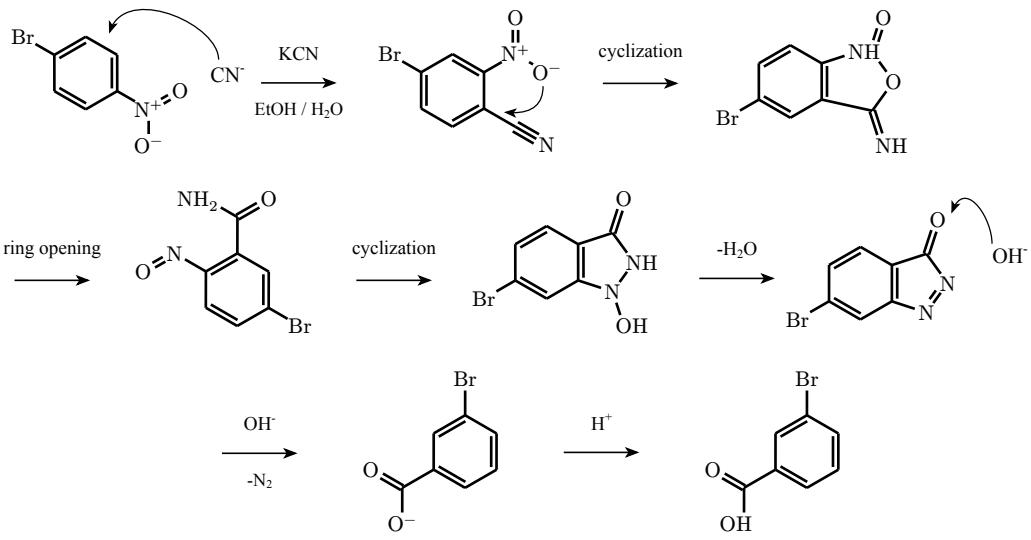
Long mechanisms can combine SMILES rendering, `#cetz-annotation` for electron movement, and ordinary CeTZ layout. This example shows the classical conversion of p-bromonitrobenzene to m-bromobenzoic acid.

```
// Electron-flow helpers.
#let electron-arrows(body) = cetz-annotation(mol => {
  import cetz.draw: *
  body(mol)
})

#let electron-arrow(
  mol,
  from,
  to,
  from-offset: (0, 0),
  to-offset: (0, 0),
  controls: ((0.35, 0.6), (0.35, 0.6)),
) = {
  import cetz.draw: *
  let start = (to: (name: mol, anchor: from), rel: from-offset)
  let end = (to: (name: mol, anchor: to), rel: to-offset)
  if controls.len() == 1 {
    bezier(
      start,
      end,
      (to: start, rel: controls.at(0)),
      stroke: 0.48pt + black,
      mark: (end: ">>", scale: 0.62, fill: black),
    )
  } else {
    bezier(
      start,
      end,
      (to: start, rel: controls.at(0)),
      (to: end, rel: controls.at(1)),
```

```
      stroke: 0.48pt + black,
      mark: (end: ">>", scale: 0.62, fill: black),
    )
  }
}

#let reagent(mol, at, label, offset: (0, 0)) = {
  import cetz.draw: *
  content(
    (to: (name: mol, anchor: at), rel: offset),
    text(size: 8pt)[#label],
    anchor: "center",
  )
}

// Reaction-arrow and row-layout helpers.
#let reaction-arrow(above: none, below: none) = cetz.canvas({
  import cetz.draw: *
  line(
    (0.08, 0),
    (1.08, 0),
    stroke: 0.6pt + black,
    mark: (end: ">>", scale: 0.68, fill: black),
  )
  if above != none {
    content((0.58, 0.3), text(size: 7.2pt)[#above], anchor: "south")
  }
  if below != none {
    content((0.58, -0.27), text(size: 6.8pt)[#below], anchor: "north")
  }
})

#let molecule(smiles, annotations: none) = box(
  width: 32mm,
  align(center)[
    #set text(size: 8pt)
    #render-smiles(
      smiles,
      abbreviate: true,
      config: (atom-sep: 1.7em),
      annotations: annotations,
    )
  ],
)

#let mechanism-row(..items) = {
  let cells = items.pos()
  grid(
    columns: cells.map(
      item => if item.at(0) == "molecule" { 32mm } else { 12mm },
```

```
    ),
    column-gutter: 0.6mm,
    align: horizon + center,
    ..cells.map(item => item.at(1)),
  )
}

// Substrate and early intermediates.
#let substrate = molecule(
  "O=[N+]([O-])c1ccc(Br)cc1",
  annotations: electron-arrows(mol => {
    import cetz.draw: *
    reagent(mol, "east", [CN#super[-]], offset: (0.52, 0.22))
    electron-arrow(
      mol,
      "east",
      "b4.50%",
      from-offset: (0.4, 0.42),
      to-offset: (0.1, 0.16),
      controls: ((0, 0.48), (0.18, 0.68)),
    )
  }),
)

#let sigma-adduct = molecule(
  "O=[N+]([O-])C1=CC(Br)=CC=C1C#N",
  annotations: electron-arrows(mol => {
    electron-arrow(
      mol,
      "a2.south",
      "b10.20%",
      from-offset: (-0.05, -0.08),
      to-offset: (0.18, 0),
      controls: ((0, -0.4),),
    )
  }),
)

#let cyclic-imidate = molecule("N=C1ON(=O)c2ccc(Br)cc21")

// Later intermediates and products.
#let nitroso-amide = molecule("O=Nc1c(C(=O)N)cc(Br)cc1")
#let hydroxy-azo = molecule("O=C1NN(O)C2=C1C=CC(Br)=C2")

#let azoketone = molecule(
  "O=C1N=NC2=C1C=CC(Br)=C2",
  annotations: electron-arrows(mol => {
    import cetz.draw: *
    reagent(mol, "east", [OH#super[-]], offset: (0.72, 0.18))
    electron-arrow(
```

```
      mol,
      "east",
      "b0.50%",
      from-offset: (0.48, 0.34),
      to-offset: (0.28, 0.38),
      controls: ((0, 0.36), (0.16, 0.5)),
    )
  }),
)

#let carboxylate = molecule("O=C([O-])c1cc(Br)ccc1")
#let product = molecule("O=C(O)c1cc(Br)ccc1")

// Compose the mechanism.
#block(breakable: false, width: 100%)[
  #stack(
    dir: ttb,
    spacing: 5mm,
    mechanism-row(
      ("molecule", substrate),
      ("arrow", reaction-arrow(above: [KCN], below: [EtOH / H#sub[2]O])),
      ("molecule", sigma-adduct),
      ("arrow", move(dy: -2.1mm, reaction-arrow(above: [cyclization]))),
      ("molecule", cyclic-imidate),
    ),
    mechanism-row(
      ("arrow", reaction-arrow(above: [ring opening])),
      ("molecule", nitroso-amide),
      ("arrow", reaction-arrow(above: [cyclization])),
      ("molecule", hydroxy-azo),
      ("arrow", reaction-arrow(above: [-H#sub[2]O])),
      ("molecule", azoketone),
    ),
    align(center)[
      #mechanism-row(
        ("arrow", reaction-arrow(above: [OH#super[-]], below: [-N#sub[2]])),
        ("molecule", carboxylate),
        ("arrow", move(dy: -2mm, reaction-arrow(above: [H#super[+]]))),
        ("molecule", product),
      )
    ],
  )
]
```

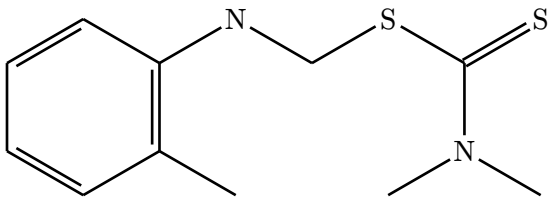


The mechanistic sequence follows M. Rosenblum, *The Mechanism of the von Richter Reaction*, J. Am. Chem. Soc. 82 (1960), 3796-3798, doi:10.1021/ja01499a090[4]. Molecule-specific anchors and curved-arrow routing can be adjusted directly in the figure source.

V.6 Low-Level Labels

Use `#label-annotation` when no leader line is needed.

```
#let mol-data = read("DepositedStructure_SUBSTANCE_SID_93298_Version_3.sdf")

#render-mol(
  mol-data,
  skeletal: true,
  annotations: label-annotation(
    molecule-anchor(anchor: "south"),
    [PubChem SID 93298],
    offset: (0, -0.65),
    label-anchor: "north",
  ),
)
```



PubChem SID 93298

[4] https://doi.org/10.1021/ja01499a090

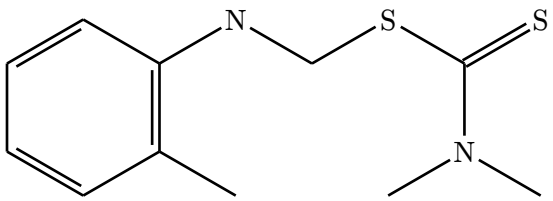
V.7 Custom CeTZ Overlays

For final figure polishing, `#cetz-annotation` exposes the generated molecule name, so you can draw directly against CeTZ anchors.

```
#let mol-data = read("DepositedStructure_SUBSTANCE_SID_93298_Version_3.sdf")

#render-mol(
  mol-data,
  skeletal: true,
  annotations: cetz-annotation(mol => {
    import cetz.draw: *
    content(
      (to: (name: mol, anchor: "north"), rel: (0, 0.54)),
      text(size: 0.82em)[database structure],
      anchor: "south",
    )
  }),
)
```

---

database structure



Part VI

Dump Mode

With `⟨dump⟩`, `molchemist` returns generated `alchemist` source instead of a drawing. Use this when a figure needs manual surgery beyond the annotation API.

```
#let mol-data = read("Structure2D_COMPOUND_CID_241.sdf")
#render-mol(mol-data, skeletal: true, dump: true)
```

```
#let base-sep = 3em
#skeletize({
  hook("a0")
  branch({
    double(absolute: -29.997863113888922deg, atom-sep: base-sep *
    1.2346525655928402, offset: "right", name: "b0")
    hook("a1")
    single(absolute: -90deg, atom-sep: base-sep * 1.2345728085928085,
    name: "b3")
    hook("a3")
    double(absolute: -150.0021368861111deg, atom-sep: base-sep *
    1.2346525655928402, offset: "right", name: "b7")
    hook("a5")
    single(absolute: 149.99927221917264deg, atom-sep: base-sep *
    1.2345456476922463, name: "b9")
    hook("a4")
    double(absolute: 90deg, atom-sep: base-sep * 1.2345728085928085,
    offset: "right", name: "b5")
    hook("a2")
    branch({
      single(absolute: 0deg, atom-sep: 0pt, stroke: none, name: "a2-
      links", links: (
      "a0": single(absolute: 30.000727780827372deg, atom-sep: base-sep *
      1.2345456476922463, name: "b1"),
    ))
    })
  })
})
```

Part VII

Publication Guidance

For paper figures, start with `⟨skeletal⟩` for hydrocarbon-heavy structures and `⟨abbreviate⟩` when heteroatom hydrogens or terminal groups should remain explicit. Keep full mode for small structures and debugging.

Keep annotations sparse. In most cases, a thin unboxed `#callout-annotation` is better than a boxed label or an arrowhead. If a leader line looks like a chemical bond, move the label with `⟨label-at⟩` or stop the leader outside the structure with `⟨leader-end⟩`.

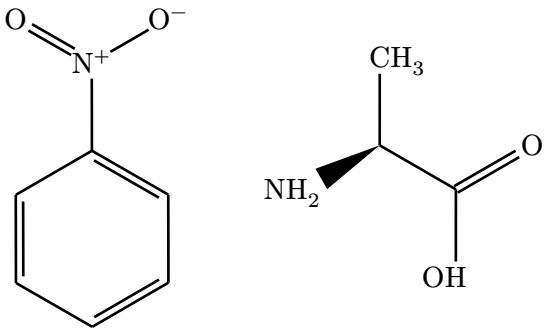
> Dense structures can still overlap in full mode, especially after SMILES implicit hydrogens are expanded. Prefer abbreviated or skeletal mode when a molecule is meant for a publication figure.

Part VIII

SMILES Notes

SMILES support is a parse-and-layout pipeline. The parser accepts common SMILES notation, aromatic rings, charges, tetrahedral `@` / `@@` centers, and `/` / `\` double-bond geometry.

```
#grid(
  columns: 2,
  gutter: 10mm,
  align: horizon + center,
  render-smiles("O=[N+]([O-])c1ccccc1", abbreviate: true),
  render-smiles("N[C@@H](C)C(=O)O", abbreviate: true),
)
```



Extended OpenSMILES chirality classes such as `@AL`, `@SP`, `@TB`, and `@OH` are accepted, but they are currently preserved as textual stereo annotations rather than native geometric depictions.

Part IX

API Reference

IX.1 Rendering Functions

```
#render-mol(
  (data),
  (abbreviate): false,
  (skeletal): false,
  (dump): false,
  (config): (:),
  (annotations): none,
  (show-indices): false
) → content
```

Render a molecule from raw Molfile or SDF text.

Argument

(data) — str | bytes | path

Raw .mol or .sdf data. Typst 0.15.0 and later may pass path(...) directly; older versions should pass read(...) output.

Argument

(abbreviate): false — bool

Enables abbreviated rendering.

Argument

(skeletal): false — bool

Enables skeletal rendering. This overrides (abbreviate).

Argument

(dump): false — bool

Returns generated alchemist source code instead of rendering the molecule.

Argument

(config): (:) — dictionary

Visual configuration passed to alchemist.

Argument

(annotations): none — annotation | array | none

Optional overlay annotation or array of annotations.

Argument

`(show-indices): false` `bool` | `str`

Debug overlay for annotation authoring. Use `true`, `"all"`, `"atoms"`, or `"bonds"`.

```
#render-smiles(
  (smiles),
  (abbreviate): false,
  (skeletal): false,
  (dump): false,
  (config): (:),
  (annotations): none,
  (show-indices): false
) → content
```

Parse a SMILES string, generate 2D coordinates, and render the resulting molecule.

Argument

`(smiles)` `str`

A SMILES string.

Argument

`(abbreviate): false` `bool`

Enables abbreviated rendering after layout generation.

Argument

`(skeletal): false` `bool`

Enables skeletal rendering. This overrides `(abbreviate)`.

Argument

`(dump): false` `bool`

Returns generated `alchemist` source code instead of rendering the molecule.

Argument

`(config): (:)` `dictionary`

Visual configuration passed to `alchemist`.

Argument

`(annotations): none` `annotation` | `array` | `none`

Optional overlay annotation or array of annotations.

Argument

`(show-indices): false` `bool` | `str`

Debug overlay for annotation authoring. Use `true`, `"all"`, `"atoms"`, or `"bonds"`.

IX.2 Anchor Helpers

#atom-anchor(⟨index⟩, ⟨anchor⟩: "mid") → **anchor**

Select an atom anchor for annotation placement.

Argument

⟨index⟩ — int

Atom index shown by ⟨show-indices⟩.

Argument

⟨anchor⟩: "mid" — str

CeTZ anchor on the atom object, such as "north", "east", or "mid".

#bond-anchor(⟨index⟩, ⟨anchor⟩: "50%") → **anchor**

Select a bond anchor for annotation placement.

Argument

⟨index⟩ — int

Bond index shown by ⟨show-indices⟩.

Argument

⟨anchor⟩: "50%" — str

CeTZ anchor on the bond object.

#molecule-anchor(⟨anchor⟩: "center") → **anchor**

Select an anchor on the whole rendered molecule.

Argument

⟨anchor⟩: "center" — str

CeTZ group anchor such as "center", "north", "east", "south", or "west".

#atom-ref(⟨index⟩) → **str**

Return the internal atom anchor name as a string.

#bond-ref(⟨index⟩) → **str**

Return the internal bond anchor name as a string.

IX.3 Annotation Builders

```
#callout-annotation(
  (at),
  (label),
  (anchor): "mid",
  (side): "north-east",
  (label-at): auto,
  (leader): "curve",
  (mark): none
) → annotation
```

Build an external label with a leader line.

Argument
(at) anchor

Target anchor.

Argument
(label) content

Label content.

Argument
(side): "north-east" str

Preset label side. Supported values are "east", "west", "north", "south", and diagonal combinations.

Argument
(label-at): auto auto | dictionary

Explicit CeTZ coordinate for the label.

Argument
(leader): "curve" str

Leader style: "curve", "straight", or "elbow".

Argument
(leader-end): auto auto | dictionary

Manual CeTZ coordinate where the leader should stop.

Argument
(target-gap): 0.2 float

Clearance from the target when (leader-end) is automatic.

```
#arrow-annotation((from), (to), (label): none, (mark): (end: ">>", fill:
luma(0%))) → annotation
```

Build a free arrow overlay.

#label-annotation(⟨at⟩, ⟨label⟩, ⟨offset⟩: (0, 0.45)) → annotation

Build a free text label overlay without a leader line.

#cetz-annotation(⟨body⟩) → annotation

Run custom CeTZ code after the molecule is drawn.

Argument

⟨body⟩ function

Function receiving the generated molecule name.

Part X

Limitations and Roadmap

- Full mode can become crowded for large molecules because explicit hydrogens and atom labels occupy real page space.
- SMILES layout is generated internally and may not match the exact layout from an external chemical drawing program.
- Extended OpenSMILES chirality classes are currently preserved as textual stereo annotations rather than native geometric depictions.
- Annotation helpers are intentionally minimal. For complex figure composition, use `#cetz-annotation` or dump the generated `alchemist` code.

Part XI

License and Dependencies

`molchemist` is distributed under the MIT license. Molfile / SDF parsing is powered by `sdfrust`; SMILES parsing is based on `opensmiles`; SMILES 2D coordinate generation uses `CoordgenLibs`; rendering is handled by `alchemist` and CeTZ. See the third-party notices distributed with the package for full license details.

The example SDF files and rendered example images are attributed separately from the package code. In particular, this manual includes PubChem-derived example structures such as CID 241[5]. See `THIRD_PARTY_NOTICES.md` and `docs/assets/README.md` for source URLs and the relevant NCBI data-usage policy.

[5] https://pubchem.ncbi.nlm.nih.gov/compound/241

Part XII

Index

A

B

C

L

M

R